

How to Cancel Asynchronous Stream in C# with Examples

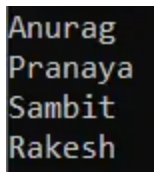
In this article, I am going to discuss **How to Cancel Asynchronous Stream in C#** with Examples. Please read our previous article where we discussed [Asynchronous Streams in C#](#) with Examples.

How to Cancel Asynchronous Stream in C#?

Here, we are going to see two ways of Cancelling an asynchronous stream. The following is the asynchronous stream example that we have created in our previous example.

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static async Task Main(string[] args)
        {
            await foreach (var name in GenerateNames())
            {
                Console.WriteLine(name);
            }
            Console.ReadKey();
        }
        private static async IEnumerable<string> GenerateNames()
        {
            yield return "Anurag";
            await Task.Delay(TimeSpan.FromSeconds(3));
            yield return "Pranaya";
            await Task.Delay(TimeSpan.FromSeconds(3));
            yield return "Sambit";
            await Task.Delay(TimeSpan.FromSeconds(3));
            yield return "Rakesh";
        }
    }
}
```

When you run the above code, you will get the following output.



```
Anurag
Pranaya
Sambit
Rakesh
```

Cancelling Asynchronous Stream in C# using Break Statement:

Now, we have one condition to break the stream. When we get the name Pranaya we need to cancel the stream. In order to do this, we need to add the break statement inside the for each loop with the if conditional statement as shown in the below image.

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static async Task Main(string[] args)
        {
            await foreach (var name in GenerateNames())
            {
                Console.WriteLine(name);
                //Some condition to break the asynchronous stream
                if (name == "Pranaya")
                {
                    break;
                }
            }
            Console.ReadKey();
        }
        private static async IAsyncEnumerable<string> GenerateNames()
        {
            yield return "Anurag";
            await Task.Delay(TimeSpan.FromSeconds(3));
            yield return "Pranaya";
            await Task.Delay(TimeSpan.FromSeconds(3));
            yield return "Sambit";
            await Task.Delay(TimeSpan.FromSeconds(3));
            yield return "Rakesh";
        }
    }
}
```

When you run the above code, you will get the following output.

```
Anurag
Pranaya
```

Canceling Asynchronous Stream in C# using Cancellation Token:

Now, we have another condition to cancel the asynchronous stream. The condition is after 5 seconds we need to cancel the stream. For this, we need to use Cancellation Token. The following shows how to use Cancellation Token to cancel an asynchronous stream in C#. The following code is self-explained, so please go through the comment lines.

```
using System;
```

```

using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static async Task Main(string[] args)
        {
            //Create an instance of CancellationTokenSource
            var CTS = new CancellationTokenSource();
            //Set the time when the token is going to cancel the stream
            CTS.CancelAfter(TimeSpan.FromSeconds(5));
            try
            {
                //Pass the Cancellation Token to GenerateNames method
                await foreach (var name in GenerateNames(CTS.Token))
                {
                    Console.WriteLine(name);
                }
            }
            catch (Exception ex)
            {
                Console.WriteLine(ex.Message);
            }
            finally
            {
                //Dispose the CancellationTokenSource
                CTS.Dispose();
                CTS = null;
            }
            Console.ReadKey();
        }
        //This method accepts Cancellation Token as input parameter
        //Set its value to default
        private static async IEnumerable<string> GenerateNames(Cancellation
token = default)
        {
            //Check if request comes for Token Cancellation
            //if(token.IsCancellationRequested)
            //{
            //    token.ThrowIfCancellationRequested();
            //}
            //But here we just need to pass the token to Task.Delay method
            yield return "Anurag";
            await Task.Delay(TimeSpan.FromSeconds(3), token);
            yield return "Pranaya";
            await Task.Delay(TimeSpan.FromSeconds(3), token);
            yield return "Sambit";
            await Task.Delay(TimeSpan.FromSeconds(3), token);
            yield return "Rakesh";
        }
    }
}

```

```
    }  
  }  
}
```

Output:

```
Anurag  
Pranaya  
A task was canceled.
```

If you see the compiler is giving some warning message in our GenerateNames method. This is because we are not using the enumerator cancellation attribute. Let us see how to fix this.

Canceling Through IAsyncEnumerable – EnumeratorCancellation in C#:

In the previous example, we saw that we were able to pass a cancellation token to our asynchronous stream. But we got one warning saying that we should be using an EnumeratorCancellation attribute in our cancellation token so that we can cancel the asynchronous stream from our IAsyncEnumerable return type.

What does that mean? Let us visualize this with an example. Let us create a method that will consume the result of the GenerateNames method as shown in the below image. Here, the ProcessNames method takes IAsyncEnumerable as a parameter, and as it is an Enumerable so we can process it using a for each loop which is shown in the below code. So, here we are processing the stream using for each loop.

```
private static async Task ProcessNames(IAsyncEnumerable<string> namesEnumerable)  
{  
    await foreach (var name in namesEnumerable)  
    {  
        Console.WriteLine(name + "Processed");  
    }  
}
```

Then from the main method, we can call this ProcessNames method as shown in the below image. Here, first, we call the GenerateNames method which will return an IAsyncEnumerable and then we pass that Enumerable to the ProcessNames method and it will work. Here we are receiving an IAsyncEnumerable when we call the GenerateNames method. This is just a representation of the stream, but we are not running the stream here. We run this stream when we access the values using a for each loop which we have done inside the ProcessNames method.

```
static async Task Main(string[] args)  
{  
    var namesEnumerable = GenerateNames();  
    await ProcessNames(namesEnumerable);  
    Console.ReadKey();  
}
```

The complete example code is given below.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static async Task Main(string[] args)
        {
            var namesEnumerable = GenerateNames();
            await ProcessNames(namesEnumerable);
            Console.ReadKey();
        }
        private static async Task ProcessNames(IAsyncEnumerable<string> names)
        {
            await foreach (var name in namesEnumerable)
            {
                Console.WriteLine($"{name} - Processed");
            }
        }
        private static async IAyncEnumerable<string> GenerateNames(Cancellation
token = default)
        {
            yield return "Anurag";
            await Task.Delay(TimeSpan.FromSeconds(3), token);
            yield return "Pranaya";
            await Task.Delay(TimeSpan.FromSeconds(3), token);
            yield return "Sambit";
            await Task.Delay(TimeSpan.FromSeconds(3), token);
            yield return "Rakesh";
        }
    }
}
```

Output:

```
Anurag - Processed
Pranaya - Processed
Sambit - Processed
Rakesh - Processed
```

And you can see, this is working. But there is just one problem. And the problem is that we cannot cancel that asynchronous stream. Why is that? Because, we never passed the cancellation token to the GenerateNames method, and that is easy to fix. But what happens if we want to pass a cancellation token from our ProcessedNames method? What happens when we want to cancel our asynchronous stream from the place where we are consuming the IAyncEnumerable stream?

To do so, we need to use the WithCancellation method of the IAsyncEnumerable as shown in the below code. So, here we are creating an instance of CancellationTokenSource and then Setting the time interval when the token is going to be canceled i.e. after 5 seconds. Then passing the Cancellation Token using the WithCancellation method.

```
private static async Task ProcessNames(IAsyncEnumerable<string> namesEnumerable)
{
    //Creating the CancellationTokenSource instance
    var CTS = new CancellationTokenSource();

    //Setting the time interval when the token is going to be cancelled
    CTS.CancelAfter(TimeSpan.FromSeconds(5));

    //Iterating the IAsyncEnumerable
    //Passing the Cancellation Token using WithCancellation method
    await foreach (var name in namesEnumerable.WithCancellation(CTS.Token))
    {
        Console.WriteLine($"{name} - Processed");
    }
}
```

With the above changes if you run the application, then also it will not work. Let us see that. Following is the complete example code as of now.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static async Task Main(string[] args)
        {
            //Here we are receiving an IAsyncEnumerable.
            //This is just a representation of the stream,
            //But we are not running the stream here
            var namesEnumerable = GenerateNames();
            await ProcessNames(namesEnumerable);
            Console.ReadKey();
        }
        private static async Task ProcessNames(IAsyncEnumerable<string> namesEnumerable)
        {
            //Creating the CancellationTokenSource instance
            var CTS = new CancellationTokenSource();
            //Setting the time interval when the token is going to be cancelled
            CTS.CancelAfter(TimeSpan.FromSeconds(5));
            //Iterating the IAsyncEnumerable
            //Passing the Cancellation Token using WithCancellation method
            await foreach (var name in namesEnumerable.WithCancellation(CTS.Token))
            {
                Console.WriteLine($"{name} - Processed");
            }
        }
    }
}
```

```

        Console.WriteLine($"{name} - Processed");
    }
}
private static async IEnumerable<string> GenerateNames(Cancellation
token = default)
{
    yield return "Anurag";
    await Task.Delay(TimeSpan.FromSeconds(3), token);
    yield return "Pranaya";
    await Task.Delay(TimeSpan.FromSeconds(3), token);
    yield return "Sambit";
    await Task.Delay(TimeSpan.FromSeconds(3), token);
    yield return "Rakesh";
}
}
}

```

Output:

```

Anurag - Processed
Pranaya - Processed
Sambit - Processed
Rakesh - Processed

```

See the stream is not canceled after 5 seconds. In order to cancel the stream, we need to decorate the Cancellation token with the EnumeratorCancellation attribute inside the GenerateNames method as shown in the below image. The EnumeratorCancellation belongs to System.Runtime.CompilerServices namespace so includes that namespace.

```

private static async IEnumerable<string> GenerateNames([EnumeratorCancellation] CancellationToken token = default)
{
    yield return "Anurag";
    await Task.Delay(TimeSpan.FromSeconds(3), token);
    yield return "Pranaya";
    await Task.Delay(TimeSpan.FromSeconds(3), token);
    yield return "Sambit";
    await Task.Delay(TimeSpan.FromSeconds(3), token);
    yield return "Rakesh";
}

```

With the above changes in place, it should work. Let us see that. Following is the complete example code.

```

using System;
using System.Collections.Generic;
using System.Runtime.CompilerServices;
using System.Threading;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static async Task Main(string[] args)

```

```

{
    //Here we are receiving an IAsyncEnumerable.
    //This is just a representation of the stream,
    //But we are not running the stream here
    var namesEnumerable = GenerateNames();
    await ProcessNames(namesEnumerable);
    Console.ReadKey();
}
private static async Task ProcessNames(IAsyncEnumerable<string> names)
{
    //Creating the CancellationTokenSource instance
    var CTS = new CancellationTokenSource();
    //Setting the time interval when the token is going to be cancelled
    CTS.CancelAfter(TimeSpan.FromSeconds(5));
    //Iterating the IAsyncEnumerable
    //Passing the Cancellation Token using WithCancellation method
    await foreach (var name in namesEnumerable.WithCancellation(CTS.Token))
    {
        Console.WriteLine($"{name} - Processed");
    }
}
private static async IAsyncEnumerable<string>
GenerateNames([EnumeratorCancellation] CancellationToken token = default)
{
    yield return "Anurag";
    await Task.Delay(TimeSpan.FromSeconds(3), token);
    yield return "Pranaya";
    await Task.Delay(TimeSpan.FromSeconds(3), token);
    yield return "Sambit";
    await Task.Delay(TimeSpan.FromSeconds(3), token);
    yield return "Rakesh";
}
}
}

```

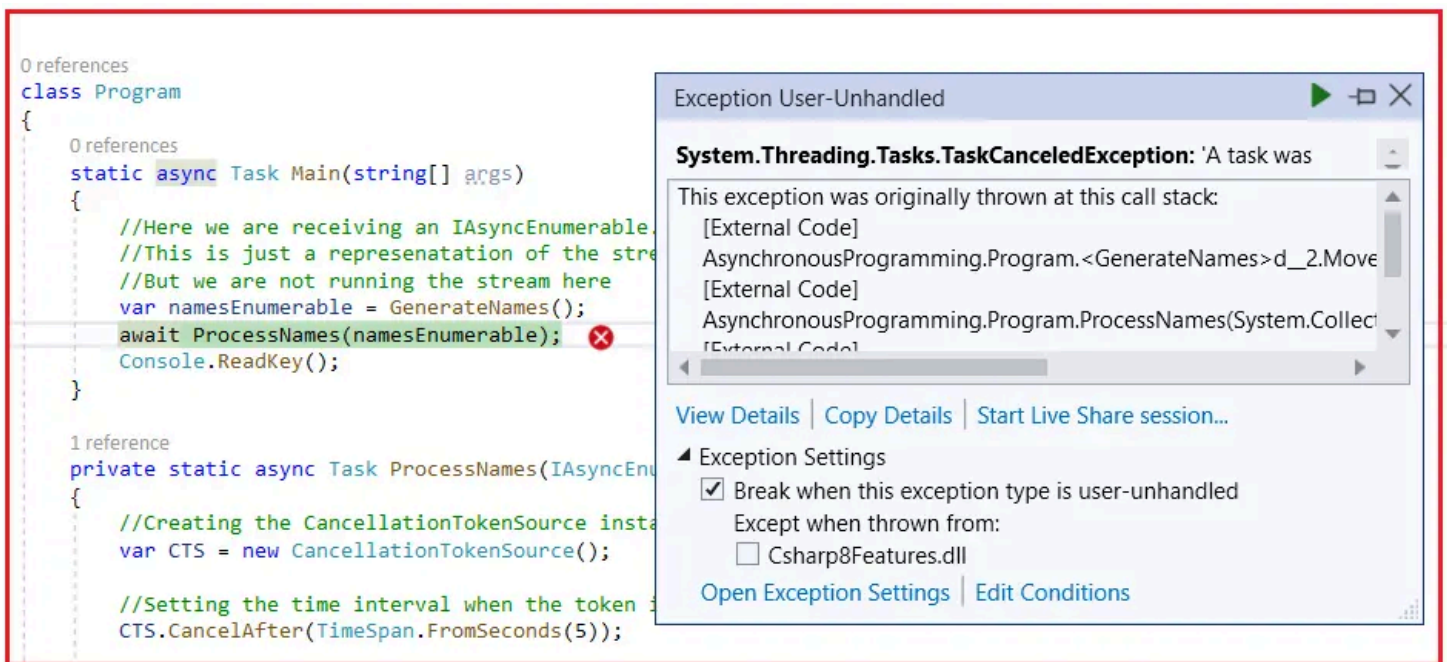
Output:

```

Anurag - Processed
Pranaya - Processed

```

So, when you run the above code, after processing the first two names it will throw the following exception. This is because we have not handled the exception.



Now, let us handle the exception and rerun the code and observe the output. Please modify the code as follows.

```
using System;
using System.Collections.Generic;
using System.Runtime.CompilerServices;
using System.Threading;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static async Task Main(string[] args)
        {
            var namesEnumerable = GenerateNames();
            await ProcessNames(namesEnumerable);
            Console.ReadKey();
        }

        private static async Task ProcessNames(IAsyncEnumerable<string> names)
        {
            var CTS = new CancellationTokenSource();
            CTS.CancelAfter(TimeSpan.FromSeconds(5));
            try
            {
                await foreach (var name in namesEnumerable.WithCancellation(CTS.Token))
                {
                    Console.WriteLine($"{name} - Processed");
                }
            }
            catch (Exception ex)
            {
                Console.WriteLine(ex.Message);
            }
        }
    }
}
```

```

    }
    finally
    {
        CTS.Dispose();
        CTS = null;
    }
}
private static async IEnumerable<string>
GenerateNames([EnumeratorCancellation] CancellationToken token = default)
{
    yield return "Anurag";
    await Task.Delay(TimeSpan.FromSeconds(3), token);
    yield return "Pranaya";
    await Task.Delay(TimeSpan.FromSeconds(3), token);
    yield return "Sambit";
    await Task.Delay(TimeSpan.FromSeconds(3), token);
    yield return "Rakesh";
}
}
}

```

Output:

```

Anurag - Processed
Pranaya - Processed
A task was canceled.

```

So, by using the `EnumeratorCancellation` attribute we are able to cancel the asynchronous stream in C#.

In the next article, I am going to discuss **Anti Patterns in Asynchronous Programming** with Examples. Here, in this article, I try to explain **How to Cancel Asynchronous Stream in C#** with Examples. I hope you enjoy this **How to Cancel Asynchronous Stream in C# with Examples** article.